

Entregable 1. Código fuente del productor y del consumidor

Integrantes: Derly Dayana Garcia Carrillo · Orlando Rojas Narvaez | Docente: Juan Carlos Polania Cortes | Periodo 2026-2
| Repositorio: github.com/OrlandoRojasNa/taller02-rabbitmq-docker

Resumen

Ambos servicios están escritos en **Java 21 con Spring Boot 3.5** y usan **Spring AMQP** para hablar con RabbitMQ. Cada uno se compila dentro de su propia imagen Docker.

Servicio	Clase	Responsabilidad
productor	MensajeController	Expone POST /api/mensajes y responde {"status": true, "message": "Mensaje encolado"}.
	PublicadorService	Valida que el cuerpo sea un objeto JSON, lo publica como mensaje persistente y espera la confirmación del broker.
	ManejadorErrores	Convierte los errores en 400 (mensaje inválido) o 503 (RabbitMQ no disponible) con status: false.
	Respuesta	record Respuesta(boolean status, String message) : garantiza que status sea booleano.
	RabbitConfig	Declara la cola durable cola.sensores (idempotente).
consumidor	MensajeListener	Lee la cola, registra en el log el contenido y la hora de recepción, y confirma con basicAck solo después de procesar.
	RabbitConfig	Declara la misma cola durable.

El productor incluye pruebas unitarias (MensajeControllerTest) que verifican el mensaje válido, los inválidos y el caso de broker caído.

Producer

producer/src/main/java/co/edu/fet/sd/producer/ProducerApplication.java

```
package co.edu.fet.sd.producer;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ProducerApplication {

    public static void main(String[] args) {
        SpringApplication.run(ProducerApplication.class, args);
    }
}
```

producer/src/main/java/co/edu/fet/sd/producer/config/RabbitConfig.java

```
package co.edu.fet.sd.producer.config;

import org.springframework.amqp.core.Queue;
import org.springframework.amqp.core.QueueBuilder;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class RabbitConfig {

    /**
     * La cola ya viene creada por definitions.json al arrancar RabbitMQ.
     * Declararla aqui tambien es idempotente (mismos atributos) y sirve de respaldo.
     */
    @Bean
    public Queue colaSensores(@Value("${app.colas}") String nombreCola) {
        return QueueBuilder.durable(nombreCola).build();
    }
}
```

producer/src/main/java/co/edu/fet/sd/producer/controller/ManejadorErrores.java

```
package co.edu.fet.sd.producer.controller;

import co.edu.fet.sd.producer.dto.Respuesta;
import co.edu.fet.sd.producer.service.MensajeInvalidoException;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.amqp.AmqpException;
import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.HttpRequestMethodNotSupportedException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

/** Toda respuesta de error sale en JSON con el mismo formato {status, message}. */
@RestControllerAdvice
```

```

public class ManejadorErrores {

    private static final Logger log = LoggerFactory.getLogger(ManejadorErrores.class);

    @ExceptionHandler(MensajeInvalidoException.class)
    public ResponseEntity<Respuesta> mensajeInvalido(MensajeInvalidoException e) {
        log.warn("Mensaje rechazado, no se publica: {}", e.getMessage());
        return responder(HttpStatus.BAD_REQUEST, "Formato de mensaje inválido");
    }

    @ExceptionHandler(AmqpException.class)
    public ResponseEntity<Respuesta> brokerNoDisponible(AmqpException e) {
        log.error("No se pudo publicar en RabbitMQ: {}", e.getMessage());
        return responder(HttpStatus.SERVICE_UNAVAILABLE, "No se pudo publicar el mensaje en la cola");
    }

    @ExceptionHandler(HttpRequestMethodNotSupportedException.class)
    public ResponseEntity<Respuesta> metodoNoSoportado(HttpRequestMethodNotSupportedException e) {
        return responder(HttpStatus.METHOD_NOT_ALLOWED, "Método no permitido, use POST");
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<Respuesta> inesperado(Exception e) {
        log.error("Error inesperado", e);
        return responder(HttpStatus.INTERNAL_SERVER_ERROR, "Error interno al procesar el mensaje");
    }

    private ResponseEntity<Respuesta> responder(HttpStatus estado, String mensaje) {
        return ResponseEntity.status(estado)
            .contentType(MediaType.APPLICATION_JSON)
            .body(Respuesta.error(mensaje));
    }
}

```

productor/src/main/java/co/edu/fet/sd/productor/controller/MensajeController.java

```

package co.edu.fet.sd.productor.controller;

import co.edu.fet.sd.productor.dto.Respuesta;
import co.edu.fet.sd.productor.service.PublicadorService;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/mensajes")
public class MensajeController {

    private final PublicadorService publicador;

    public MensajeController(PublicadorService publicador) {
        this.publicador = publicador;
    }

    /**
     * Recibe el cuerpo como texto crudo para validar nosotros mismos el JSON

```

```

    * y poder responder siempre con el formato {status, message}.
    */
    @PostMapping(produces = MediaType.APPLICATION_JSON_VALUE)
    public ResponseEntity<Respuesta> publicar(@RequestBody(required = false) String cuerpo) {
        publicador.publicar(cuerpo);
        return ResponseEntity.ok(Respuesta.ok("Mensaje encolado"));
    }
}

```

productor/src/main/java/co/edu/fet/sd/productor/dto/Respuesta.java

```

package co.edu.fet.sd.productor.dto;

/**
 * Formato de respuesta de la API. {@code status} es boolean, asi Jackson lo
 * serializa como true/false y no como la cadena "true".
 */
public record Respuesta(boolean status, String message) {

    public static Respuesta ok(String message) {
        return new Respuesta(true, message);
    }

    public static Respuesta error(String message) {
        return new Respuesta(false, message);
    }
}

```

productor/src/main/java/co/edu/fet/sd/productor/service/MensajeInvalidoException.java

```

package co.edu.fet.sd.productor.service;

public class MensajeInvalidoException extends RuntimeException {

    public MensajeInvalidoException(String detalle) {
        super(detalle);
    }
}

```

productor/src/main/java/co/edu/fet/sd/productor/service/PublicadorService.java

```

package co.edu.fet.sd.productor.service;

import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.databind.DeserializationFeature;
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.ObjectReader;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.amqp.core.Message;
import org.springframework.amqp.core.MessageBuilder;
import org.springframework.amqp.core.MessageDeliveryMode;
import org.springframework.amqp.core.MessageProperties;
import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

import java.nio.charset.StandardCharsets;
import java.util.Date;

```

```

import java.util.UUID;

@Service
public class PublicadorService {

    private static final Logger log = LoggerFactory.getLogger(PublicadorService.class);

    private final RabbitTemplate rabbitTemplate;
    private final ObjectReader lectorJson;
    private final String cola;
    private final long confirmTimeoutMs;

    public PublicadorService(RabbitTemplate rabbitTemplate,
                            ObjectMapper objectMapper,
                            @Value("${app.cola}") String cola,
                            @Value("${app.confirm-timeout-ms}") long confirmTimeoutMs) {
        this.rabbitTemplate = rabbitTemplate;
        // Rechaza cuerpos como '{"a":1} basura' (contenido extra despues del JSON)
        this.lectorJson =
objectMapper.reader().with(DeserializationFeature.FAIL_ON_TRAILING_TOKENS);
        this.cola = cola;
        this.confirmTimeoutMs = confirmTimeoutMs;
    }

    /**
     * Valida que el cuerpo sea un objeto JSON y lo publica como mensaje persistente.
     * Si no es valido lanza {@link MensajeInvalidoException} y no se publica nada.
     */
    public String publicar(String cuerpo) {
        JsonNode json = validar(cuerpo);

        String id = UUID.randomUUID().toString();
        Message mensaje = MessageBuilder
            .withBody(json.toString().getBytes(StandardCharsets.UTF_8))
            .setContentType(MessageProperties.CONTENT_TYPE_JSON)
            .setContentEncoding(StandardCharsets.UTF_8.name())
            // Persistente: sobrevive a un reinicio del broker (junto con la cola durable)
            .setDeliveryMode(MessageDeliveryMode.PERSISTENT)
            .setMessageId(id)
            .setTimestamp(new Date())
            .build();

        // Publica por el exchange por defecto (routing key = nombre de la cola)
        // y espera la confirmacion del broker antes de responder.
        rabbitTemplate.invoke(operaciones -> {
            operaciones.send("", cola, mensaje);
            operaciones.waitForConfirmsOrDie(confirmTimeoutMs);
            return null;
        });

        log.info("Mensaje publicado en '{}' id={} contenido={}", cola, id, json);
        return id;
    }

    private JsonNode validar(String cuerpo) {
        if (cuerpo == null || cuerpo.isBlank()) {
            throw new MensajeInvalidoException("cuerpo vacio");
        }
        JsonNode json;
        try {
            json = lectorJson.readTree(cuerpo);
        }
    }

```

```

    } catch (JsonProcessingException e) {
        throw new MensajeInvalidoException("JSON mal formado: " + e.getOriginalMessage());
    }
    if (json == null || !json.isObject() || json.isEmpty()) {
        throw new MensajeInvalidoException("se esperaba un objeto JSON con al menos un
campo");
    }
    return json;
}
}

```

productor/src/test/java/co/edu/fet/sd/productor/controller/MensajeControllerTest.java

```

package co.edu.fet.sd.productor.controller;

import co.edu.fet.sd.productor.service.PublicadorService;
import org.junit.jupiter.api.Test;
import org.springframework.amqp.AmqpConnectException;
import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.context.annotation.Import;
import org.springframework.http.MediaType;
import org.springframework.test.context.TestPropertySource;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;

import java.net.ConnectException;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.never;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.content;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@WebMvcTest(MensajeController.class)
@Import({PublicadorService.class, ManejadorErrores.class})
@TestPropertySource(properties = {"app.colas=cola.test", "app.confirm-timeout-ms=1000"})
class MensajeControllerTest {

    @Autowired
    private MockMvc mvc;

    @MockitoBean
    private RabbitTemplate rabbitTemplate;

    @Test
    void mensajeValidoSeEncolaYStatusEsBoolean() throws Exception {
        mvc.perform(post("/api/mensajes")
            .contentType(MediaType.APPLICATION_JSON)
            .content("{\"sensor\": \"0D-01\", \"valor\": 4.2, \"unidad\": \"mg/L\"}"))
            .andExpect(status().isOk())
            .andExpect(content().contentType(MediaType.APPLICATION_JSON))
            .andExpect(jsonPath("$.status").value(true))
            .andExpect(jsonPath("$.status").isBoolean())
            .andExpect(jsonPath("$.message").value("Mensaje encolado"));

        verify(rabbitTemplate).invoke(any());
    }
}

```

```

}

@Test
void jsonMalFormadoNoSePublica() throws Exception {
    mvc.perform(post("/api/mensajes")
        .contentType(MediaType.APPLICATION_JSON)
        .content("{\"sensor\": \"0D-01\", \"valor\": }"))
        .andExpect(status().isBadRequest())
        .andExpect(content().contentType(MediaType.APPLICATION_JSON))
        .andExpect(jsonPath("$.status").value(false))
        .andExpect(jsonPath("$.status").isBoolean())
        .andExpect(jsonPath("$.message").value("Formato de mensaje inválido"));

    verify(rabbitTemplate, never()).invoke(any());
}

@Test
void textoPlanoNoSePublica() throws Exception {
    mvc.perform(post("/api/mensajes")
        .contentType(MediaType.TEXT_PLAIN)
        .content("esto no es json"))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.status").value(false));

    verify(rabbitTemplate, never()).invoke(any());
}

@Test
void cuerpoVacioNoSePublica() throws Exception {
    mvc.perform(post("/api/mensajes").contentType(MediaType.APPLICATION_JSON))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.status").value(false));

    verify(rabbitTemplate, never()).invoke(any());
}

@Test
void jsonConBasuraAlFinalNoSePublica() throws Exception {
    mvc.perform(post("/api/mensajes")
        .contentType(MediaType.APPLICATION_JSON)
        .content("{\"sensor\": \"0D-01\"} basura"))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.status").value(false));

    verify(rabbitTemplate, never()).invoke(any());
}

@Test
void brokerCaidoRespondeStatusFalse() throws Exception {
    when(rabbitTemplate.invoke(any())).thenThrow(new AmqpConnectException(new
ConnectException("sin broker")));

    mvc.perform(post("/api/mensajes")
        .contentType(MediaType.APPLICATION_JSON)
        .content("{\"sensor\": \"0D-01\"}"))
        .andExpect(status().isServiceUnavailable())
        .andExpect(jsonPath("$.status").value(false));
}
}

```

producer/src/main/resources/application.yml

```
server:
  port: 8080

spring:
  application:
    name: producer
  rabbitmq:
    host: ${SPRING_RABBITMQ_HOST:localhost}
    port: ${SPRING_RABBITMQ_PORT:5672}
    username: ${SPRING_RABBITMQ_USERNAME:lab}
    password: ${SPRING_RABBITMQ_PASSWORD:lab123}
    # El broker confirma cada publicacion: solo respondemos "encolado" cuando RabbitMQ lo acepto
    publisher-confirm-type: simple
    connection-timeout: 5s

app:
  cola: ${APP_COLA:cola.sensores}
  confirm-timeout-ms: 5000
```

Consumidor

consumidor/src/main/java/co/edu/fet/sd/consumidor/ConsumidorApplication.java

```
package co.edu.fet.sd.consumidor;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ConsumidorApplication {

    public static void main(String[] args) {
        SpringApplication.run(ConsumidorApplication.class, args);
    }
}
```

consumidor/src/main/java/co/edu/fet/sd/consumidor/config/RabbitConfig.java

```
package co.edu.fet.sd.consumidor.config;

import org.springframework.amqp.core.Queue;
import org.springframework.amqp.core.QueueBuilder;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class RabbitConfig {

    /** Misma declaracion que el productor y que definitions.json: cola durable. */
    @Bean
    public Queue colaSensores(@Value("${app.colas}") String nombreCola) {
        return QueueBuilder.durable(nombreCola).build();
    }
}
```

consumidor/src/main/java/co/edu/fet/sd/consumidor/listener/MensajeListener.java

```
package co.edu.fet.sd.consumidor.listener;

import com.rabbitmq.client.Channel;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.amqp.core.Message;
import org.springframework.amqp.core.MessageProperties;
import org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.stereotype.Component;

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.time.ZonedDateTime;
import java.time.format.DateTimeFormatter;
import java.util.concurrent.atomic.AtomicLong;

@Component
public class MensajeListener {
```

```

private static final Logger log = LoggerFactory.getLogger(MensajeListener.class);
private static final DateTimeFormatter FORMATO_HORA = DateTimeFormatter.ofPattern("yyyy-MM-dd
HH:mm:ss.SSS XXX");

private final AtomicLong contador = new AtomicLong();

@RabbitListener(queues = "${app.colas}")
public void recibir(Message mensaje, Channel canal) throws IOException {
    MessageProperties props = mensaje.getMessageProperties();
    long deliveryTag = props.getDeliveryTag();

    try {
        String contenido = new String(mensaje.getBody(), StandardCharsets.UTF_8);
        String horaRecepcion = ZonedDateTime.now().format(FORMATO_HORA);

        log.info("[#{}] Mensaje recibido | hora de recepcion: {} | id: {} | reentregado: {} |
contenido: {}",
                contador.incrementAndGet(), horaRecepcion, props.getMessageId(),
                props.isRedelivered(), contenido);

        // Solo despues de procesarlo se confirma (ACK) y RabbitMQ lo retira de la cola
        canal.basicAck(deliveryTag, false);
    } catch (Exception e) {
        log.error("Error procesando el mensaje, se devuelve a la cola", e);
        canal.basicNack(deliveryTag, false, true);
    }
}
}

```

consumidor/src/main/resources/application.yml

```

spring:
  application:
    name: consumidor
  main:
    web-application-type: none
  rabbitmq:
    host: ${SPRING_RABBITMQ_HOST:localhost}
    port: ${SPRING_RABBITMQ_PORT:5672}
    username: ${SPRING_RABBITMQ_USERNAME:lab}
    password: ${SPRING_RABBITMQ_PASSWORD:lab123}
    listener:
      simple:
        # ACK manual: el mensaje solo sale de la cola despues de procesarlo
        acknowledge-mode: manual
        prefetch: 1
        # Si RabbitMQ aun no esta listo, el listener reintenta en lugar de terminar con error
        missing-queues-fatal: false
        retry:
          enabled: false

  app:
    cola: ${APP_COLA:cola.sensores}

  logging:
    pattern:
      console: "%d{yyyy-MM-dd HH:mm:ss.SSS} %-5level %logger{0} - %msg%n"

```